

Unit 1 · Topic Notes

Wireframing, UI/UX Planning & Project Scoping

Course: Full Stack Application Development with MERN

Learning Objectives

1. Create low-fidelity wireframes before writing any React code
 2. Apply core UI/UX principles (hierarchy, consistency, feedback, accessibility)
 3. Scope an MVP using the MoSCoW method
 4. Avoid the classic “design while coding” trap
-

1. Why Wireframe Before Coding?

Think about a time you started building a page in React, then had to redo the whole layout two days later — it happens to nearly everyone. A wireframe costs 10 minutes on paper; a layout rebuild in JSX costs hours. This is risk reduction, not busywork.

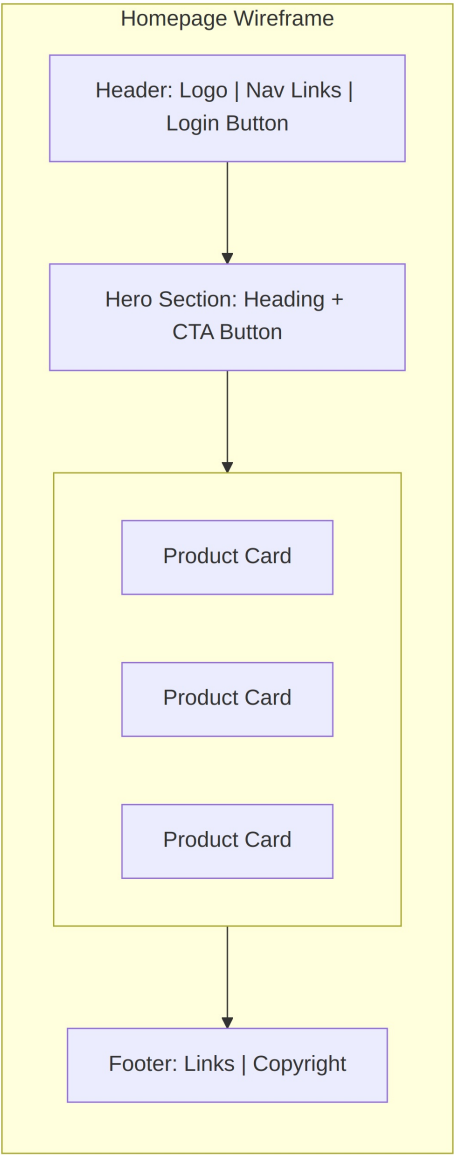
2. Low-Fidelity vs. High-Fidelity

	Low-fidelity	High-fidelity
Tools	Pen & paper, Excalidraw	Figma, Adobe XD
Detail	Boxes, labels, rough layout	Real colors, fonts, spacing
Speed	Minutes	Hours
When to use	Before build, for structure decisions	Before build, for client sign-off / polish

For this course: low-fidelity is mandatory before any component code is written. High-fidelity is optional/bonus.

3. Reading a Wireframe as Layout Regions

Represent page structure as labeled regions — this maps 1:1 to how you’ll later build it with CSS Grid/Flexbox in React:



Wireframe layout regions diagram

Each box here becomes a React component later (`<Header />` , `<Hero />` , `<ProductCard />` , `<Footer />`) — wireframing regions *is* component planning.

4. Core UI/UX Principles

Principle	Bad example	Good example
Visual hierarchy	Every element same font size	Heading > subheading > body, clear size steps
Consistency	3 different button styles on one page	One primary button style reused everywhere
Feedback	Button does nothing visible when clicked	Loading spinner / disabled state on click
Accessibility	Grey text on white background	Sufficient color contrast, alt text on images

5. Project Scoping with MoSCoW

Once wireframes exist, prioritize features before coding starts:

--	--	--

Category	Meaning	Example (capstone: “campus lost & found app”)
Must have	App fails without it	Post a lost item, browse items, search
Should have	Important, not launch-blocking	Email notification when a match is found
Could have	Nice bonus if time allows	Image upload with drag-and-drop
Won’t have (this version)	Explicitly deferred	Chat between finder and owner

This directly feeds the “Out of scope” section from the requirements topic — same discipline, different lens.

6. Defining an MVP

MVP = the smallest version of the app that still delivers real value and can be demoed end-to-end. For a student team, the MVP checklist: - Covers 1 full user journey start to finish (not 5 half-built ones) - Uses only the “Must have” list from MoSCoW - Can be demonstrated live in under 3 minutes

7. Hands-On Lab (35 min, pairs or teams of 3)

Using the same scenario as the requirements topic (*campus infrastructure reporting app*):

1. Sketch a low-fi wireframe (paper, Excalidraw, or a Mermaid region diagram like §3) for:
 - The “Report an issue” screen
 - The “Admin dashboard” screen
2. List 3 UI/UX principle violations in the example screenshot shown in class, and how you’d fix each
3. Run a MoSCoW pass on your 6 user stories from the previous topic — sort each into Must/Should/Could/Won’t
4. Write your one-sentence MVP definition

8. Quick Assessment

1. What’s the main difference between low-fi and high-fi wireframes?
2. Give an example of “feedback” as a UI principle, other than a loading spinner.
3. In MoSCoW, where would “dark mode” usually go for a student project, and why?
4. Why does wireframing help *before* writing React components specifically?
5. What are the two criteria for a good MVP?

Notes prepared for Full Stack Learning — MERN course, Unit 1.